OpenShift Resource Tuning Guide

our-nservice | our-namespace

Version 2.0 — Updated with OOM Root Cause Analysis

Prepared by	Payments Engineering Team
Version	2.0 (updated March 2026 — OOM findings incorporated)
Clusters	dc1 (dc00 + dc11) — Active-Active
Namespace	our-namespace
Data sources	Grafana TARC dashboard, Datadog Kubernetes Pods, Container logs

> **!** This document was updated after container log analysis revealed the root cause of all 10 pod restarts. The original memory assessment ("healthy, no action needed") has been revised. Sections 3, 4, 5, and 7 contain new or significantly changed content.

1. Executive Summary

This document provides a complete analysis of our-nservice's OpenShift resource configuration and a team-executable remediation plan covering three scenarios: high-volume days, datacenter unavailability, and cost efficiency through right-sizing.

Version 2.0 incorporates root cause findings from container log analysis that materially change the remediation priority and sequence.

Original findings (v1.0)

- CPU requests over-provisioned by 7x (534m requested vs ~74m actual), causing 80.1% namespace quota usage.
- HPA not enabled — no automatic scaling during traffic spikes or datacenter drain.
- Memory assessed as healthy at 63% average utilization. This assessment was incorrect.

New findings (v2.0)

All 10 pod restarts in dc11 datacenter share a single root cause confirmed by log analysis:

- Root cause: OutOfMemoryError: Java heap space triggered by a Hazelcast degradation event at ~14:20 UTC.
- Amplifier: A blocking CompletableFuture.get() call at PaymentService.java:148 inside a reactive boundedElastic thread causes thread pool explosion (300–430 threads per pod) when Hazelcast is slow.
- Configuration defect: MaxRAMPercentage=25 gives the JVM only ~448Mi heap on a 1792Mi app container — far too small for a payments service with reactive pipelines and distributed caching.

- Active risk: Pod 4wsk2 (dc11) is currently live with 430 boundedElastic threads — exceeding the count at which x8mx5 crashed. It is at elevated risk of the same failure on the next Hazelcast event.

> **!** The blocking .get() call at PaymentService.java:148 is the root amplifier. Without fixing this line, any Hazelcast degradation — however brief — will repeat the cascade regardless of heap size increases or HPA configuration.

2. Current State Baseline

All data sourced from Grafana TARC dashboard, Datadog Kubernetes Pods, and container log analysis on March 14, 2026.

2.1 Infrastructure topology

Property	Value
Deployment	our-nservice
Namespace	our-namespace
Cluster	dc1
Datacenters	dc00 (Dc00anta) + dc11 (Dc111) — active-active
Pods per DC	24 (48 total across both DCs)
Containers per pod	4 (1 app + 3 sidecars: Envoy proxy, log agent, security)
App container limit	1792Mi (confirmed from deployment env vars)
Pod status	All 48 running, 0 failed, 0 pending
HPA status	Not enabled

2.2 CPU — current vs actual

Metric	Value	Notes
cpu.request per pod	534m	Scheduled reservation
cpu.limit per pod	1800m	Hard ceiling
Actual CPU per pod	~74m	4.1% of limit (Datadog)
Over-provision ratio	7.2x	Request is 7x actual
Total requests (24 pods)	12.8 cores	Consumed from quota
Namespace CPU quota	16.0 cores	Platform ceiling
Quota utilization	80.1%	Yellow — at risk
CPU throttling (mean)	0.492s/min	~74µs per pod — negligible

2.3 Memory — revised assessment

Metric	Value	Notes
App container mem limit	1792Mi	Confirmed from deployment config
JVM MaxRAMPercentage	25%	Yields only ~448Mi heap
Effective max JVM heap	~448Mi	Critically undersized
Observed avg mem usage	~1.46 GiB (63%)	Time-averaged — masks spikes
mem.request per pod	2.08 GiB	Includes all 4 containers
mem.limit per pod	2.31 GiB	Namespace-level gauge (all containers)
Namespace mem quota used	30.9%	Quota headroom is fine
Memory assessment (v1)	INCORRECT	OOM restarts disprove it

Note The 63% average memory utilization figure from Datadog reflects the time-averaged value across 24 hours. OOM events that last 30-90 seconds during a traffic or cache spike barely move the average. Average metrics are insufficient for detecting JVM heap pressure — use JVM-specific heap metrics instead.

2.4 Pod restarts — root cause confirmed

Metric	Value	Notes
dc00 restarts	7	Not yet log-analyzed — likely same root cause
dc11 restarts	10	Root cause confirmed: OOM Java heap space
Exit code	3	-XX:ExitOnOutOfMemoryError already configured
Crash window	14:21-15:01 UTC	9 pods in 40 minutes + 1 follow-on at 18:24
Shared trigger	Hazelcast degradation ~14:20 UTC	Confirmed by log correlation
Pod 4wsk2 status	ACTIVE RISK — 430 threads	Exceeds crash threshold

3. Root Cause Analysis — OOM Cascade

This section documents the complete failure chain derived from container log analysis. All findings are evidence-based.

3.1 Crash timeline

The tight clustering of 9 crashes between 14:21 and 15:01 UTC is definitive evidence of a shared external trigger rather than independent pod failures.

Pod	OOM Time (UTC)	Age at Query	Restarts
x8mx5	~14:21 UTC	6h 3m ago	1
m9djs	~14:36 UTC	5h 48m ago	1
7z84t	~14:42 UTC	5h 42m ago	1
x84tv	~14:42 UTC	5h 42m ago	1
5fb5s (1st)	~14:46 UTC	—	1 of 2
f68pk	~14:50 UTC	5h 34m ago	1
jxbgc	~14:56 UTC	5h 28m ago	1
xxw6c	~14:56 UTC	5h 28m ago	1
dlvpt	~15:01 UTC	5h 23m ago	1
5fb5s (2nd)	~18:24 UTC	120m ago	2 total

3.2 The four-stage failure chain

Stage 1 — Hazelcast degrades at ~14:20 UTC

All pods show a surge of Hazelcast circuit breaker warnings immediately before their respective OOM. All four cache operations begin failing simultaneously:

- enrollmentDataCache.getAsync — 100ms timeout, firing continuously
- tokenCache.getAsync — 100ms timeout, firing continuously
- paymentCache.getAsync — timed out, returning null
- updateDataInCache — failing

Every cache operation that previously returned in microseconds now takes the full 100ms timeout on every call, across every pod simultaneously.

Stage 2 — Blocking call amplifies the damage (PaymentService.java:148)

The m9djs pod surfaced this stack trace immediately before its crash:

```
java.util.concurrent.TimeoutException
  at
PaymentService.lambda$getPaymentsForNonMigratingMember$2(PaymentService.java:148)
  ...CompletableFuture.timedGet(CompletableFuture.java:1960)
```

PaymentService.java line 148 is calling CompletableFuture.get() — a blocking call inside a reactive boundedElastic thread. When Hazelcast is slow, this call blocks the thread for the full 100ms timeout duration instead of releasing it back to the pool.

Why this is critical	In Project Reactor, blocking a boundedElastic thread tells the scheduler the thread is busy. The scheduler responds by spawning new threads to handle incoming requests. With Hazelcast timing out on every call, threads accumulate faster than they complete. The pool grows without bound.

Stage 3 — boundedElastic thread pool grows uncontrolled

Thread IDs in the logs confirm the pool explosion. Each thread holds references to its full execution context: reactive pipeline state, request/response objects, Reactor context, stack traces from timeout exceptions, and Hazelcast error objects.

Pod	Peak boundedElastic threads	Status
x8mx5	311	Crashed
5fb5s (2nd crash)	139	Crashed
4wsk2 (LIVE — 17h uptime)	430	ACTIVE RISK

Stage 4 — Heap exhausted, JVM exits with code 3

Each stalled thread holds approximately 1-2 MB of retained heap objects. At 300+ threads, cumulative retention of ~450 MB exceeds the 448Mi heap ceiling. GC cannot reclaim objects still referenced by live stalled threads. -XX:ExitOnOutOfMemoryError (already configured) fires immediately, producing the log line:

```
Terminating due to java.lang.OutOfMemoryError: Java heap space
```

OpenShift restarts the container. All in-flight payment requests on that pod are lost.

3.3 Memory configuration root cause

The JVM is using UseContainerSupport (correct for containers) but the MaxRAMPercentage default of 25% was never overridden. This produces a critically undersized heap:

```
App container limit:    1792Mi
MaxRAMPercentage:          25%   (default — never explicitly set)
Effective max heap:      ~448Mi

300 stalled threads x ~1.5MB each = ~450MB heap retained
450MB  >=  448Mi heap ceiling  ->  OutOfMemoryError
```

Insight	At 25% MaxRAMPercentage, 300 stalled threads alone consume the entire heap before a single business object is allocated. Even at normal traffic with no Hazelcast issues, 448Mi is dangerously small for a Spring Boot app with Hazelcast client, reactive pipelines, token caching, and payment processing logic.

4. Risk Analysis

4.1 Active risk — pod 4wsk2 (dc11)

Pod 4wsk2 is currently running with 430 boundedElastic threads — more than x8mx5 had when it crashed at 311 threads. This pod is being sustained only by the absence of another Hazelcast degradation event. Any further cache instability will tip it into OOM.

Immediate action	Capture a thread dump and heap histogram from pod 4wsk2 before it crashes and the evidence is lost. See Section 5.1 for the exact commands.

4.2 High-volume days

During peak payment processing (month-end, bill pay cycles), two compounding risks exist:

- Higher request volume increases the rate at which boundedElastic threads are spawned when any downstream dependency (Hazelcast, database, external service) is slow.
- HPA is not enabled, so pod count is static at 24 per DC. Under load, there is no automatic relief. Latency rises rather than pod count.

With the code fix and HPA in place, the system scales out to absorb load before threads accumulate to dangerous levels.

4.3 Datacenter drain

When one DC is drained, the surviving DC serves 100% of production traffic with its existing 24 pods. Per-pod load doubles. Without HPA, this is unabsorbable. The drain scenario also doubles the rate of Hazelcast calls per pod, increasing the probability of triggering the thread accumulation pattern.

Scenario	Value	Status
Normal state CPU/pod	~74m (4.1%)	Normal half-traffic
Drain state CPU/pod	~148m (8.2%)	Doubles on single DC
HPA trigger (60% of 200m)	120m	Fires before saturation
HPA scales to	~30 pods	Absorbs drain load
Quota at 30 pods x 200m	6.0 cores	37.5% of 16-core quota — safe
Quota at 30 pods x 534m (now)	16.02 cores — OVER	HPA stalls at pod 29

4.4 Cost efficiency

Metric	Value	Notes
Actual CPU usage (all 24 pods)	~1.6 cores	12.5% of what is requested
CPU request total today	12.8 cores	80.1% of namespace quota
CPU request total after fix	4.8 cores	30% of namespace quota

Metric	Value	Notes
Quota freed	+8.0 cores	Returned to shared pool
Max HPA pods (current 534m)	30	Before quota ceiling
Max HPA pods (target 200m)	80	After right-sizing

5. Remediation Plan

Seq	Steps must be applied in the order shown. Do not enable HPA (Step 5) until Steps 1-4 are deployed and stable. Enabling HPA on a service with an active OOM condition scales instability, not capacity.

Step 0 — Capture thread dump from pod 4wsk2 (do this now)

Pod 4wsk2 is at 430 threads and at risk of crashing. Capturing a thread dump before it crashes provides the most complete picture of what objects are retaining heap and which code paths are stalling. Once it crashes, the evidence is gone.

```
# Identify the pod
oc get pods -n our-namespace | grep 4wsk2

# Capture thread dump (non-destructive)
oc exec 4wsk2 -n our-namespace -- jcmd 1 Thread.print > thread_dump_4wsk2.txt

# Capture heap histogram (shows which object types are consuming heap)
oc exec 4wsk2 -n our-namespace -- jcmd 1 GC.heap_info
oc exec 4wsk2 -n our-namespace -- jcmd 1 VM.native_memory

# Confirm current boundedElastic thread count
oc exec 4wsk2 -n our-namespace -- jcmd 1 Thread.print \
  | grep -c 'boundedElastic'

# Confirm MaxRAMPercentage in use
oc exec 4wsk2 -n our-namespace -- \
  jcmd 1 VM.flags | grep -E 'MaxRAM|HeapSize|RAM'
```

Step 1 — Raise MaxRAMPercentage from 25% to 65% (config change — deploy today)

This is a deployment environment variable change only — no code change required. It raises the JVM heap ceiling from ~448Mi to ~1.13 GiB on the current 1792Mi container, providing 2.5x more headroom to absorb thread accumulation during the next Hazelcast event while the code fix (Step 2) is being reviewed and deployed.

Current JAVA_OPTS (problematic):

```
# MaxRAMPercentage is absent or set to 25 (UseContainerSupport default)
```

```
# Result: ~448Mi max heap on a 1792Mi container
-XX:+UseContainerSupport   # this alone is NOT sufficient
```

Target JAVA_OPTS:

```
env:
  - name: JAVA_OPTS
    value: >-
      -XX:+UseContainerSupport
      -XX:MaxRAMPercentage=65.0
      -XX:InitialRAMPercentage=40.0
      -XX:MinRAMPercentage=25.0
      -XX:+UseG1GC
      -XX:+ExitOnOutOfMemoryError
      -XX:+HeapDumpOnOutOfMemoryError
      -XX:HeapDumpPath=/tmp/heapdump.hprof

# -XX:+ExitOnOutOfMemoryError is already set — confirmed by exit code 3
# -XX:+HeapDumpOnOutOfMemoryError is new — writes /tmp/heapdump.hprof on OOM
# for post-mortem analysis with Eclipse MAT or VisualVM

# Result: ~1.13 GiB max heap on 1792Mi container (65%)
# Leaves ~630Mi for: Metaspace, thread stacks, JIT cache, DD agent, sidecars
```

Metric	Before	After
Heap before	~448Mi	25% of 1792Mi
Heap after	~1.13 GiB	65% of 1792Mi
300-thread headroom before	~0 MB — instant OOM	Threads fill entire heap
300-thread headroom after	~680 MB remaining	Room for normal business objects

Step 2 — Fix blocking call at PaymentService.java:148 (code change — permanent fix)

This is the root amplifier. The blocking CompletableFuture.get() call inside a reactive boundedElastic thread is what converts a slow Hazelcast into hundreds of stalled threads. The heap fix (Step 1) raises the ceiling. This fix removes the mechanism that fills it.

Current code — the problem:

```
// PaymentService.java:148
// PROBLEM: .get() blocks the boundedElastic thread for 100ms on every Hazelcast
timeout
// Reactor responds by spawning new threads -> pool grows to 300-430 -> OOM
CompletableFuture<Data> future = hazelcastCache.getAsync(key);
Data result = future.get(100, TimeUnit.MILLISECONDS);  // <-- THIS LINE
```

Option A — Fully reactive (recommended):

```
// Convert CompletableFuture to Mono — never blocks, thread released immediately
// If Hazelcast times out, onErrorReturn provides a safe fallback
Mono<Data> result = Mono.fromCompletionStage(hazelcastCache::getAsync, key)
```

```
    .timeout(Duration.ofMillis(100))
    .onErrorReturn(fallbackValue);  // fallback: null, default object, or cache-
miss sentinel
```

Option B — If CompletableFuture.get() must be kept (lower-risk migration):

```
// Move the blocking call off the shared boundedElastic pool onto an isolated
scheduler
// This bounds blast radius — a slow Hazelcast cannot exhaust the shared pool
Scheduler isolated = Schedulers.newBoundedElastic(20, 1000, "hazelcast-blocking");

Mono<Data> result = Mono.fromCallable()
    () -> hazelcastCache.getAsync(key).get(100, TimeUnit.MILLISECONDS))
    .subscribeOn(isolated)
    .onErrorReturn(fallbackValue);
```

Recommendation	Option A is the correct long-term solution and aligns with the reactive programming model already in use. Option B is an acceptable interim step if Option A requires significant refactoring. Either option prevents the thread pool explosion.

Step 3 — Cap the boundedElastic thread pool (safety net)

Even after the code fix, adding a hard ceiling on the boundedElastic pool prevents any future blocking code from spinning up hundreds of threads silently. This is a defense-in-depth measure.

```
// Add to your Spring Boot application startup class or @Configuration bean
@PostConstruct
public void configureReactorSchedulers() {
    // Hard cap at 50 threads — prevents silent pool explosion
    // Tasks beyond the cap are queued (up to 10,000) rather than spawning new
threads
    Schedulers.setFactory(new Schedulers.Factory() {
        @Override
        public ScheduledExecutorService newBoundedElastic(
                int threadCap, int queuedTaskCap, ThreadFactory threadFactory, int
ttlSeconds) {
            return Schedulers.newBoundedElastic(50, 10000,
"boundedElastic").scheduledExecutorService;
        }
    });
}

// Add a Datadog monitor: alert when boundedElastic thread count > 30
// This gives you warning before the 50-thread cap is hit
```

Step 4 — Right-size cpu.request from 534m to 200m

Unchanged from v1.0. This is a prerequisite for HPA — without it, a datacenter drain pushes the surviving cluster over its 16-core quota ceiling exactly when HPA needs to scale up.

```
resources:
  requests:
```

```
    cpu:    "200m"      # Was 534m. Actual usage ~74m + sidecar buffer
    memory: "2.5Gi"     # Raised from 2.08 GiB to reflect true baseline + OOM
headroom
  limits:
    cpu:    "1800m"     # Unchanged
    memory: "3Gi"       # Raised from 2.31 GiB. At MaxRAMPercentage=65: ~1.95 GiB
heap
```

Raising mem.limit to 3 GiB (from 2.31 GiB) is now recommended alongside the MaxRAMPercentage fix. At 65% of 3 GiB, the heap ceiling rises to ~1.95 GiB — providing ~500 MB of spike headroom above the 1.46 GiB observed average.

Metric	Value	Notes
mem.limit per pod (new)	3 GiB	Up from 2.31 GiB
Max JVM heap at 65%	~1.95 GiB	Up from ~448Mi
Observed avg usage	~1.46 GiB	OOM spike headroom: ~500 MB
Total mem requests (24 pods)	60 GiB	Up from 49.5 GiB
Namespace quota used	37.5%	Up from 30.9% — still well within 160 GiB quota
HPA max (36 pods x 2.5 GiB)	90 GiB	56% of quota — safe at full scale

Step 5 — Add preStop hook and Spring Boot graceful shutdown

Required before enabling HPA. Without this, scale-down terminates pods mid-request, dropping in-flight payment transactions.

```
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 60   # Must exceed preStop sleep + shutdown
phase
      containers:
        - name: our-nservice
          lifecycle:
            preStop:
              exec:
                command: ["/bin/sh", "-c", "sleep 15"]

# application.properties (or application.yml):
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

Step 6 — Enable HPA (one object per datacenter)

Deploy only after Steps 1-5 are deployed and stable in both DCs. The settings below are derived from the drain scenario: the surviving DC must absorb 2x traffic and HPA must scale up before pods saturate.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: our-nservice-hpa
  namespace: our-namespace
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: our-nservice
  minReplicas: 8
  maxReplicas: 36
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60   # 60% of 200m = triggers at ~120m actual
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300   # 5 min wait — traffic redistribution
after drain
      policies:
        - type: Pods
          value: 2
          periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
        - type: Pods
          value: 4
          periodSeconds: 30
```

6. Datacenter Drain Operational Playbook

Follow these steps when draining a datacenter with HPA enabled and the fixes from Section 5 deployed.

6.1 Before the drain — pre-flight checklist

1. Confirm HPA is active: oc get hpa -n our-namespace
2. Confirm pod count is at or above minReplicas (8) in surviving DC.
3. Check namespace CPU quota: oc describe quota -n our-namespace — CPU requests should be below 50%.
4. Check boundedElastic thread count on all pods — none should be above 30.
5. Confirm Hazelcast is healthy — a drain during Hazelcast degradation risks the OOM cascade on doubled per-pod load.

6.2 During the drain

6. Route all traffic to surviving DC via load balancer / DNS failover.
7. Monitor HPA events: oc describe hpa our-nservice-hpa -n our-namespace

8. Expected: within 60-90 seconds, HPA fires scale-up. Pod count climbs from 8 toward 24-30.
9. In Datadog: CPU per pod climbs from ~74m toward ~148m. HPA threshold is 120m — scale-up fires before saturation.
10. If any pods show Pending status: check oc describe pod <name> for scheduling failure reason. Likely cause: quota exceeded (resolve by verifying cpu.request right-sizing was deployed).

6.3 After the drain — DC restoration

11. Restore traffic to drained DC gradually: 10% / 25% / 50% over 5-10 minutes.
12. Surviving DC CPU drops as traffic redistributes. HPA enters scale-down stabilization (300 second window).
13. After 5 minutes, HPA begins removing pods at 2/minute. Each pod terminates with 15s preStop + up to 30s graceful shutdown.
14. Monitor Datadog for payment transaction error rates during scale-down — any spike indicates pods are being terminated before completing in-flight requests.
15. Confirm boundedElastic thread count returns to normal (<20) on all pods after drain recovery.

7. Monitoring and Alerting Recommendations

The OOM cascade was detectable in advance — pod 4wsk2 reached 430 threads without any alert firing. These monitors close that gap.

Metric	Alert Threshold	Rationale	Priority
boundedElastic thread count	> 30 per pod	Warning at 30, critical at 45. 10-minute notice before crash threshold (50).	High
JVM heap usage %	> 80% for 5 min	Leading indicator of heap pressure before OOM occurs. Use JVM metrics from Spring Boot Actuator.	High
HPA desired != current replicas	> 5 minutes	Indicates HPA cannot schedule pods — likely a quota issue during drain.	High
Hazelcast circuit breaker	Any continuous firing	Hazelcast degradation is the root trigger. Alert immediately on sustained cache timeouts.	High
Container restarts	> 0 in 1h	Any restart is a payment-impacting event. Alert immediately, do not wait for threshold.	High

Metric	Alert Threshold	Rationale	Priority
CPU throttling	> 5s/min per pod	Rising throttling predicts latency impact before customers feel it.	Medium
CPU quota usage	> 70% of namespace	Early warning before HPA scaling is blocked by quota ceiling.	Medium
Memory quota usage	> 70% of namespace	With mem.limit raised to 3 GiB, recheck at full HPA scale (36 pods).	Low

8. Configuration Summary — Before and After

Parameter	Before	After	Impact
cpu.request per pod	534m	200m	62% reduction — quota freed
cpu.limit per pod	1800m	1800m	No change
mem.request per pod	2.08 GiB	2.5 GiB	Raised to reflect true baseline
mem.limit per pod	2.31 GiB	3 GiB	Raised for heap headroom
MaxRAMPercentage	25% (~448Mi heap)	65% (~1.95 GiB heap)	4.4x heap increase
boundedElastic cap	None (unbounded)	50 threads	Prevents silent pool explosion
PaymentService.java:148	Blocking .get()	Reactive Mono	Root amplifier removed
CPU quota used (24 pods)	12.8c / 80.1%	4.8c / 30%	50 quota points freed
Min pods per DC	24 (static)	8	HPA floor
Max pods per DC	24 (static)	36	Drain + growth ceiling
HPA enabled	No	Yes	Per-DC autoscaling
preStop / graceful shutdown	Unknown	Yes (15s)	Protects in-flight payments
Drain readiness	At risk	Ready	Full drain coverage
OOM under Hazelcast failure	Cascade crash	Graceful fallback	Code + config fix

9. Action Items and Owners

#	Action	Owner	Priority	Timing
0	Capture thread dump + heap histogram from pod 4wsk2	App team	IMMEDIATE	Now — pod at crash risk
1	Raise MaxRAMPercentage from 25% to 65% in JAVA_OPTS	App team	Critical	Deploy today — config only
2	Fix blocking CompletableFuture.get() at PaymentService.java:148	App team (dev)	Critical	Next sprint — code change
3	Cap boundedElastic pool at 50 threads in startup config	App team (dev)	Critical	With Step 2
4	Raise mem.limit to 3 GiB, mem.request to 2.5 GiB in deployment	App team	High	With Step 1
5	Right-size cpu.request from 534m to 200m	App team	High	Before HPA
6	Add preStop hook (sleep 15) + terminationGracePeriodSeconds: 60	App team	High	Before HPA
7	Set server.shutdown=graceful in application.properties	App team	High	Before HPA
8	Deploy HPA manifest to dc00 cluster	App team + Platform	High	After Steps 1-7 stable
9	Deploy HPA manifest to dc11 cluster	App team + Platform	High	After dc00 HPA validated
10	Set Datadog alert: boundedElastic threads > 30	App team	Medium	Before HPA
11	Set Datadog alert: JVM heap > 80% for 5 min	App team	Medium	Before HPA
12	Set Datadog alert: Hazelcast circuit breaker continuous fire	App team	Medium	Ongoing
13	Validate HPA scale-up in non-prod with synthetic drain test	App team	Medium	Before prod cutover
14	Analyze heap dump from any future OOM with Eclipse MAT	App team (dev)	Low	If OOM recurs

Document version 2.0 — updated March 2026 with container log root cause analysis. | Source data: Grafana TARC, Datadog Kubernetes Pods, pod container logs | Cluster: dc1